



UNIVERSIDAD MAYOR DE SAN ANDRES
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARERRA DE INFORMATICA

DEFENSA DE TESIS DE GRADO

DIGITALIZACIÓN INTELIGENTE DE RECIBOS MANUSCRITOS PARA PEQUEÑOS COMERCIANTES MEDIANTE UNA APLICACIÓN MÓVIL BASADA EN DEEP LEARNING

PARA OPTAR AL TÍTULO DE LICENCIATURA EN INFORMÁTICA
MENCIÓN: INGENIERÍA DE SISTEMAS INFORMÁTICOS

UNIV.: WHILLY EDGAR AMORAGA MAMANI
TUTOR: DR. JUAN MARCOS MIRANDA NINA PH.D.

2025



TABLA DE CONTENIDO

- **MARCO REFERENCIAL**
- **MARCO TEÓRICO**
- **MARCO APLICATIVO**
- **RESULTADOS**
- **CONCLUSIONES**





MARCO REFERENCIAL

CAPITULO I



INTRODUCCIÓN



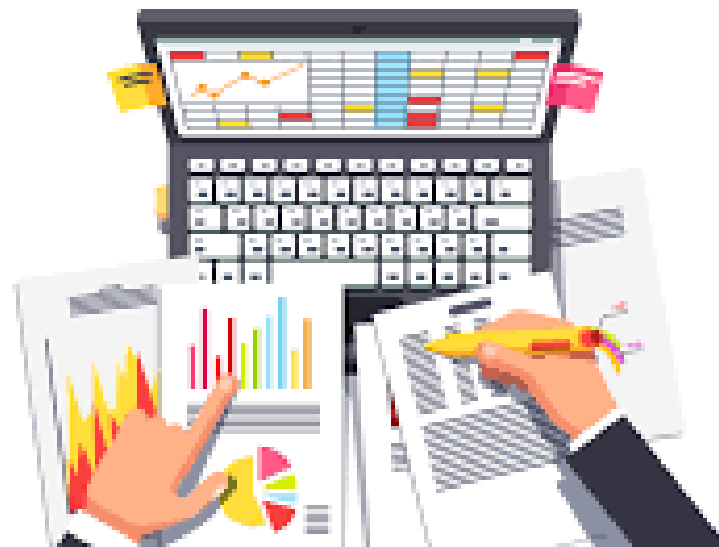
ANTENCEDENTES

TIPO	PROYECTO/TESIS	TEMA	RESULTADO
INTL.	Luz Silvana Tarazona Cruz, 2024, PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ	Reconocimiento de texto en manuscritos históricos peruanos utilizando modelos mixtos	Reconocimiento de escritura antigua con IA
NAC.	BLANCA REINA CHOQUE GARCÍA, 2015, UNIVERSIDAD MAYOR DE SAN ANDRÉS	DISEÑO DE UN AUTÓMATA PARA LA TRADUCCIÓN DEL IDIOMA CHINO AL ESPAÑOL	Aplicación práctica de OCR y lenguajes formales

PROBLEMA GENERAL

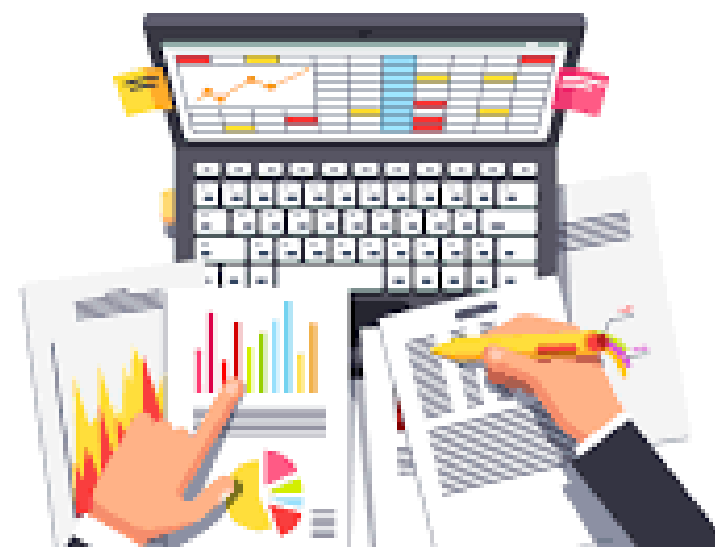
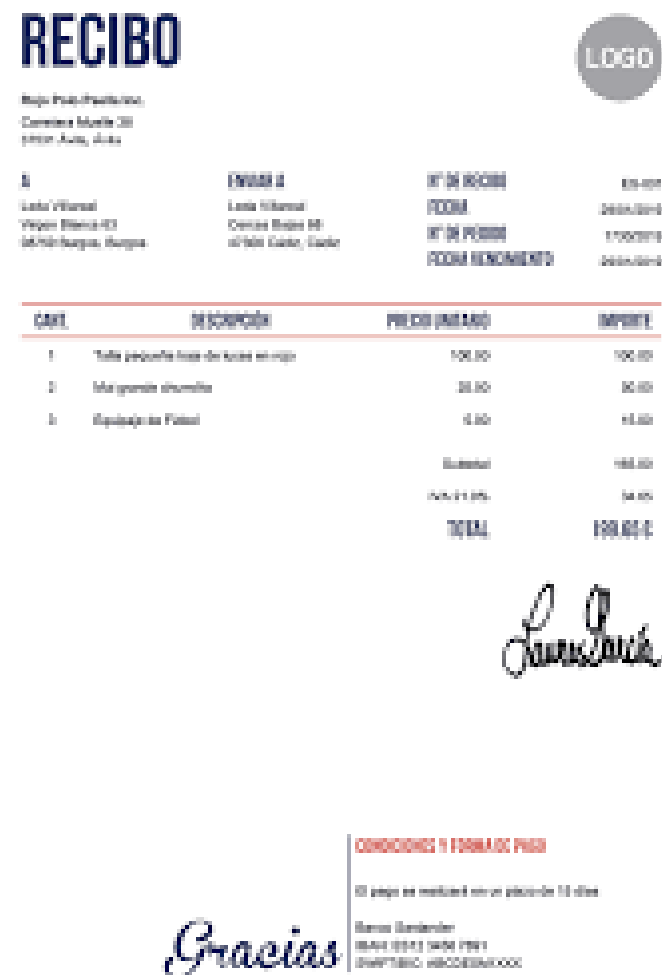


PROBLEMAS ESPECÍFICOS



OBJETIVO GENERAL

OBJETIVOS ESPECÍFICOS



HIPOTESIS

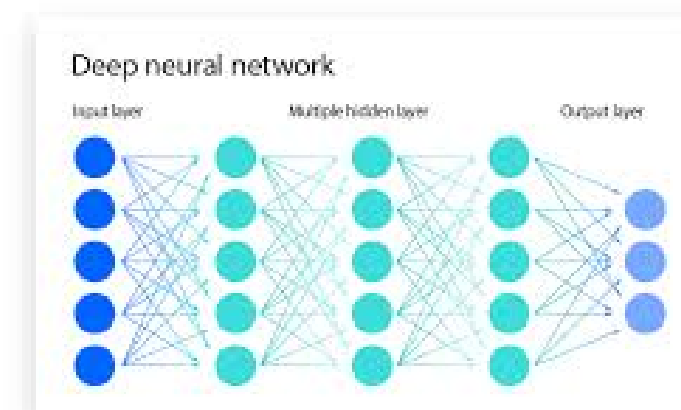
VARIABLES

DEPENDIENTE

INDEPENDIENTE

INTERVINIENTE

H => 70%



JUSTIFICACION SOCIAL



JUSTIFICACION ECONOMICA



JUSTIFICACION ACADEMICA



ALCANCES

- APLICACIÓN MÓVIL
- USO DE CRNN
- RECIBOS DIGITALES
- DATOS
ALMACENADOS
- ANÁLISIS DE DATOS
BÁSICOS
- FUNCION OFFLINE

LIMITES

- AREA LA PAZ
- RECIBOS
MANUSCRITOS
- LETRA CLARA Y
LEGIBLE
- ALMACENAMIENTOS
ESPACIO EN MÓVIL
- EFICIENCIA VARIA,
FACTORES EXTERNOS

METODOLOGIAS

- METODO CIENTIFICO
- MOBILE-D



MARCO TEORICO

CAPITULO II



CRNN



CNN

- reconocer patrones en imágenes



RNN

- toma de decisión de clasificación

YOLOv5 + Tesseract OCR



YOLOv5

- Detección de objetos



Tesseract OCR

- Reconoce texto impreso

MOBILE - D



Desarrollo Ágil Enfocado en Apps Móviles



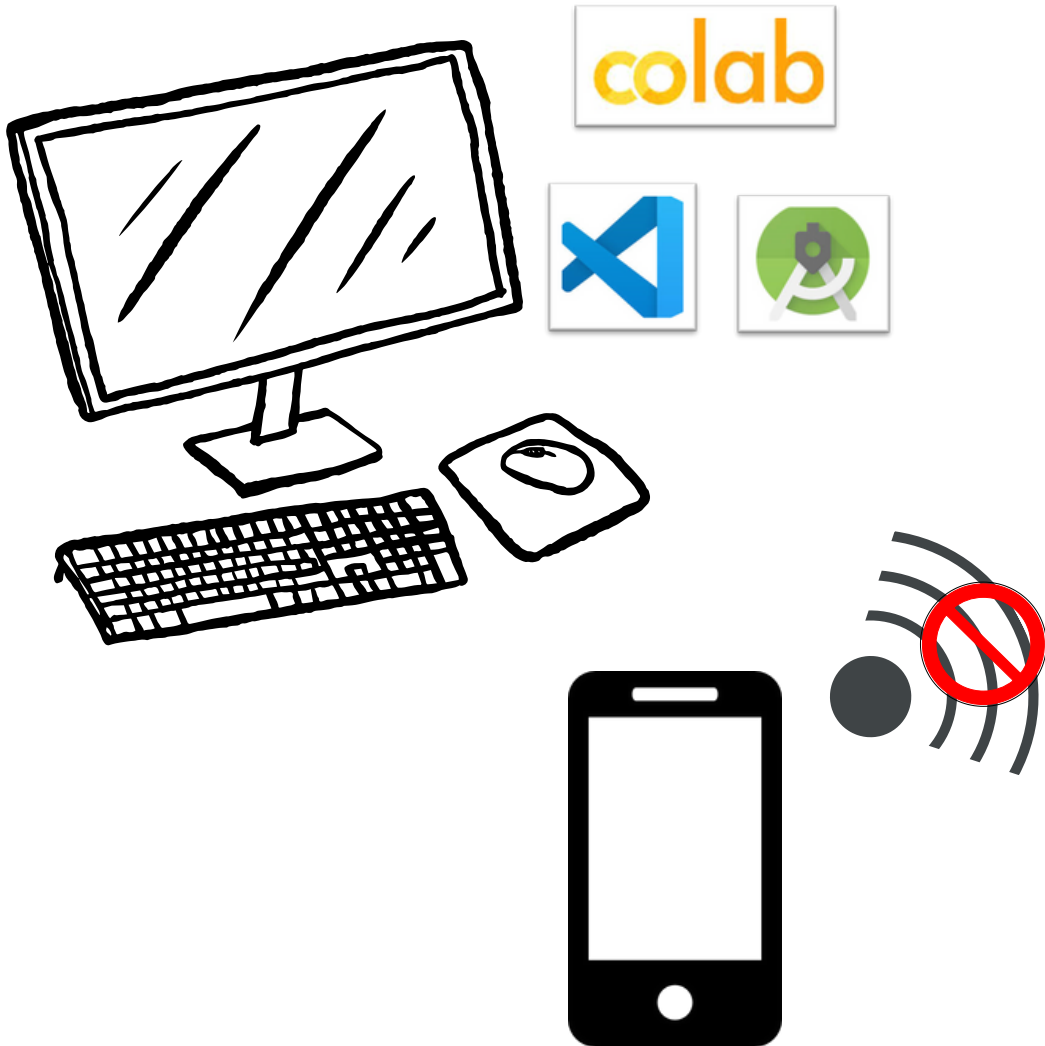
MARCO APLICATIVO

CAPITULO III



Fase de Exploracion E Inicializacion

EXPLORACION		
R. FUNCIONALES	R. NO FUNCIONALES	TECNOLOGIAS
CAPTURA DE RECIBOS	INTERFAZ INTUITIVA Y SIMPLE	YoloV5
DETECCION AREAS YOLO	PORCESAMIENTO MENOR A 15 S.	TensorFlow
RECONOCIMIENTO CRNN	FUNCIONAMIENTO OFFLINE	Github
EXPORTACION RECIBO DIGITAL	IDIOMA ESPAÑOL	Kotlin
ALAMCENAMIENTO DE DATOS		Tflite
ANALISIS DE DATOS		Sqlite



☒ Configuración de entorno de desarrollo:
VsCode, Android Studio, Colab.

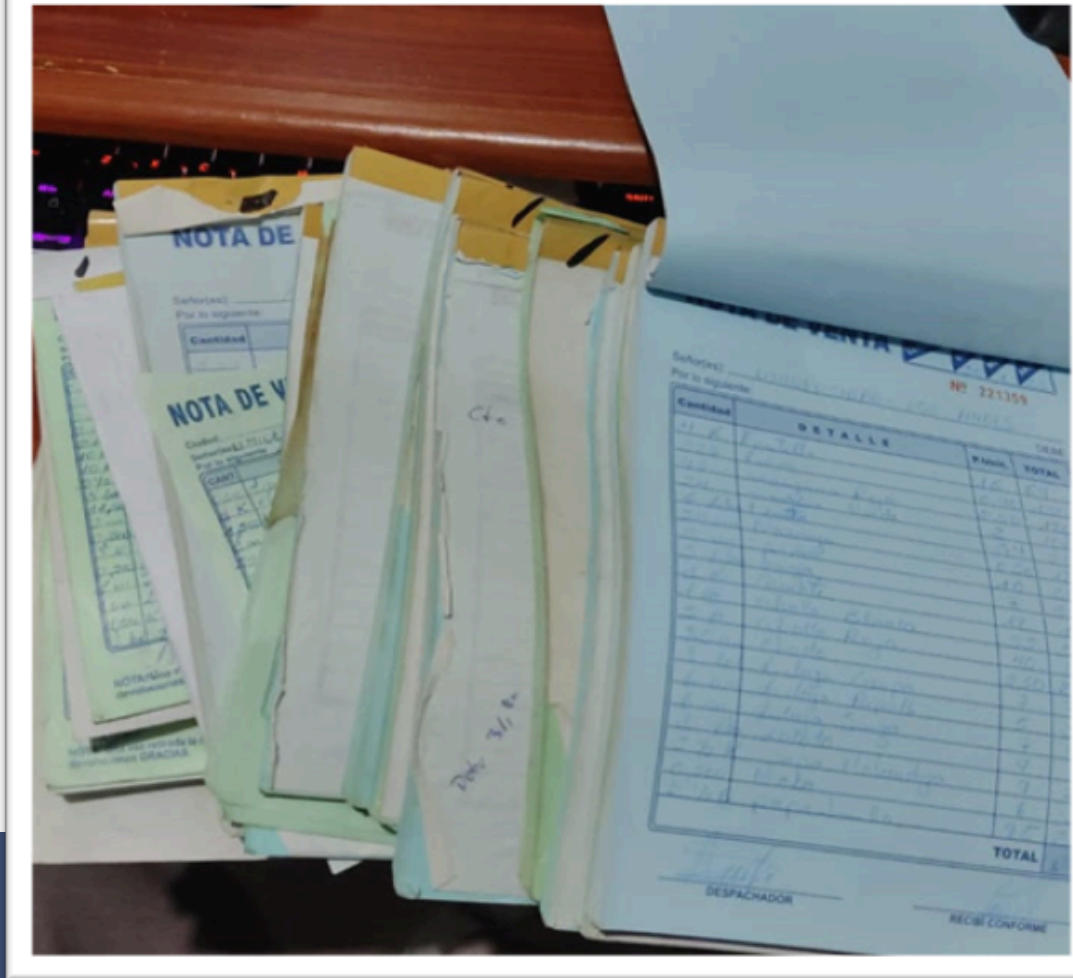
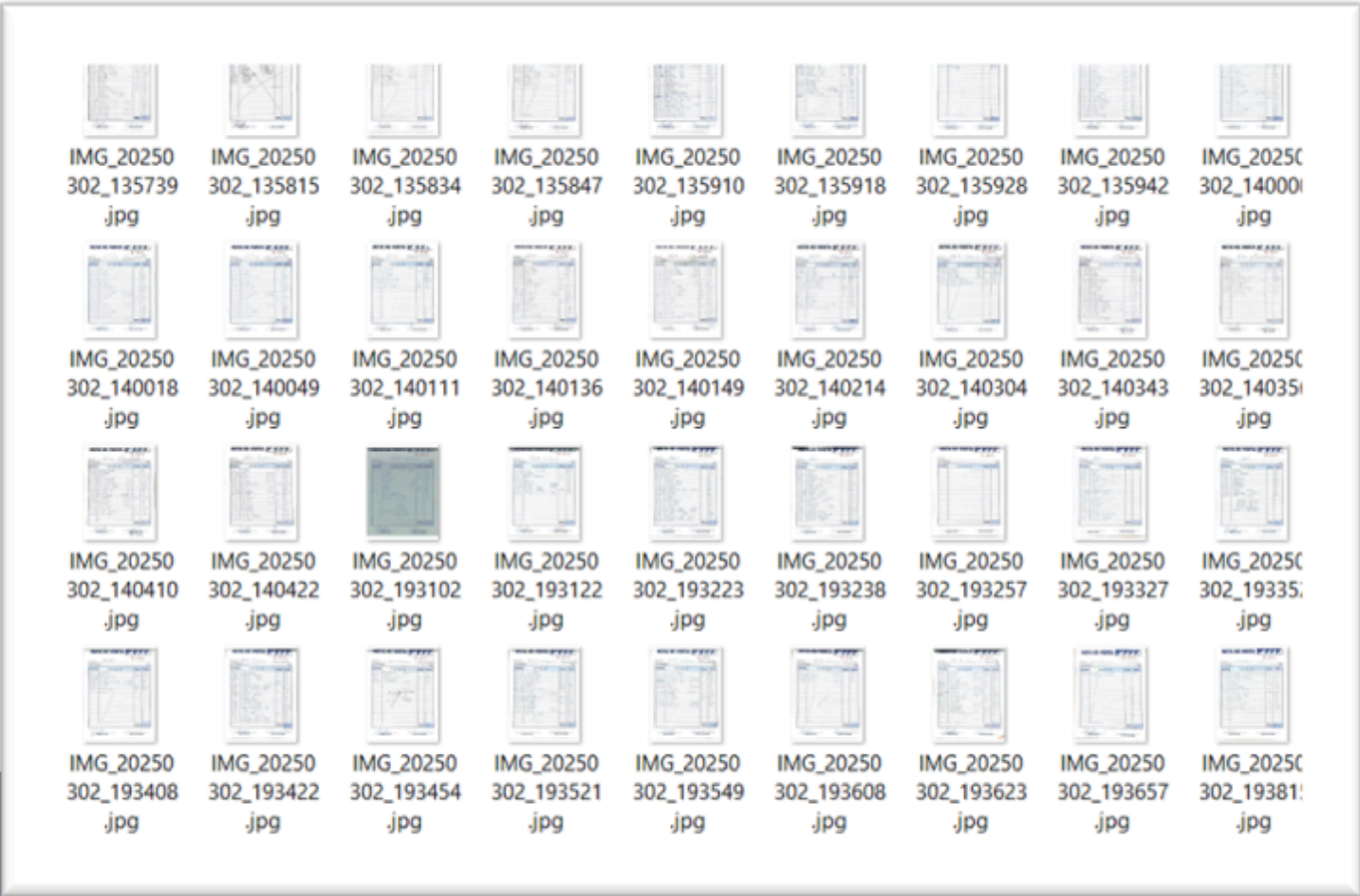
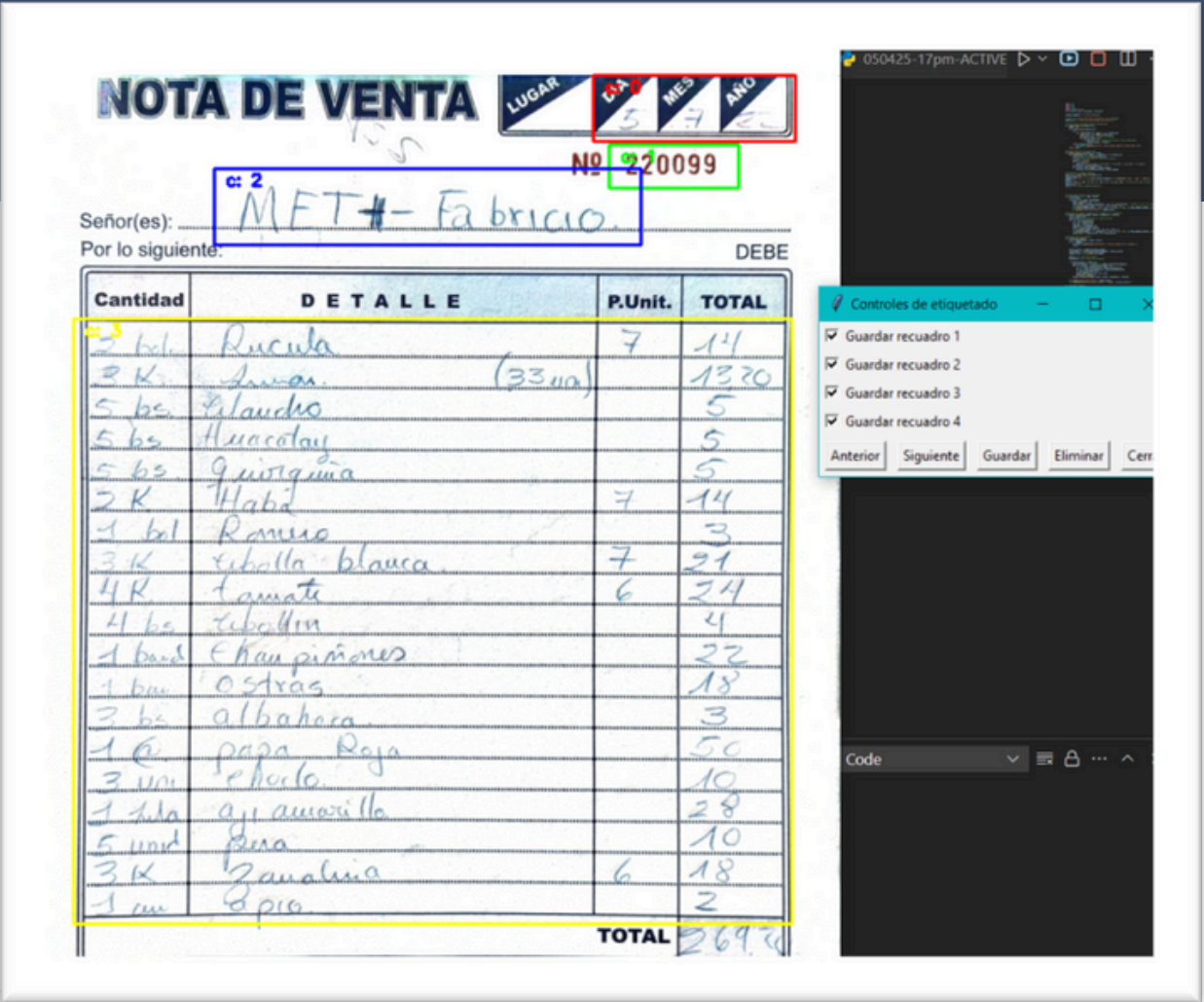


☒ Planificación de fases e iteraciones.



Fase de Produccion - Iterarion 1

- ✓ • Recopilación y captura.
- ✓ • Diseño de Script Python .
- ✓ • Detección de boundingbox.

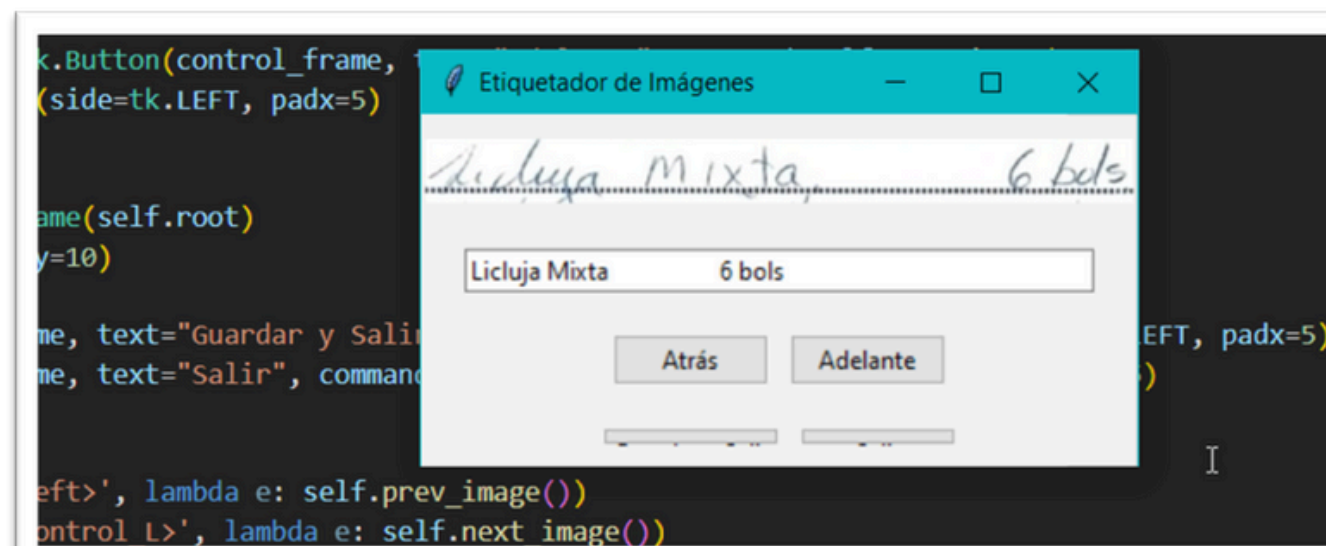


Mobile-D

Fase de Produccion - Iterarion 4

✓ • Script Python para CRNN.

✓ • Entrenamiento CRNN



```
class Config:
    data_dir = "/content/datasetCRNN-Final"
    batch_size = 64
    lr = 0.001
    epochs = 50
    img_height = 64
    img_width = 384
    device = torch.device('cuda' if torch.cuda.is_available() else
```

```
predict_batch(Config.image_folder, Config.output_txt)
```

```
✓ images/IMG_20250302_132133_box4_19.jpg => espinaca
✓ images/IMG_20250302_132133_box4_20.jpg => Frutila
✓ images/IMG_20250302_132133_box4_21.jpg => Lechuga Mixta
✓ images/IMG_20250302_132133_box4_23.jpg => Lchua Suiza (
✓ images/IMG_20250302_132133_box4_24.jpg => melan O ( 2 ur
✓ images/IMG_20250302_132133_box4_25.jpg => Nauya ( 49 uni
✓ images/IMG_20250302_132133_box4_26.jpg => palta
✓ images/IMG_20250302_132133_box4_27.jpg => papa Holandeza
```

```
class CRNN(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(128, 256, 3, padding=1), nn.BatchNorm2d(256), nn.ReLU(),
            nn.Conv2d(256, 512, 3, padding=1), nn.BatchNorm2d(512), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, None))
        )
        self.rnn = nn.LSTM(512, 256, bidirectional=True, num_layers=2)
        self.fc = nn.Linear(512, vocab_size)

    def forward(self, x):
        x = self.cnn(x)
        x = x.squeeze(2).permute(2, 0, 1)
        x, _ = self.rnn(x)
        return self.fc(x)
```

Mobile-D

Fase de Producción - Iterarion 5

- ☑ • Corrección Mediante Diccionario De Palabras
En Python. SYMSPELL
- ☑ • Estructura De Notas Digitales Mediante
Detección De Posiciones. lista bidimensional
de pares

Fase de Producción- Iterarion 6

• Conversión modelos TFLITE

```
!python export.py --weights runs/train/yolov5Class3/weights/best.pt --include tflite --data /content/yolov5_subclass3/data.yaml --img 640 --include saved_model --device cpu

export: data=/content/yolov5_subclass3/data.yaml, weights=['runs/train/yolov5Class3/weights/best.pt'], imgsz=[640], batch_size=1, device=cpu, half=False, inplace=False, keras=False
YOLOv5 v7.0-419-gcd44191c Python-3.11.12 torch-2.6.0+cu124 CPU

Fusing layers...
YOLOv5s summary: 157 layers, 7020913 parameters, 0 gradients, 15.8 GFLOPs

PyTorch: starting from runs/train/yolov5Class3/weights/best.pt with output shape (1, 25200, 9) (13.8 MB)
2025-05-24 23:50:22.898455: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when (
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1748130622.915541 29621 cuda_dnn.cc:8310] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
E0000 00:00:1748130622.920377 29621 cuda_blas.cc:1418] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered

TensorFlow SavedModel: starting export with tensorflow 2.18.0...
WARNING ⚠ using Tensorflow 2.18.0 > 2.13.1 might cause issue when exporting the model to tflite https://github.com/ultralytics/yolov5/issues/12489

from n      params module      arguments
2025-05-24 23:50:25.181428: E external/local_xla/xla/stream_executor/cuda/cuda_driver.cc:152] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: CUDA_ERROR_NO_DE
0      -1 1      3520 models.common.Conv [3, 32, 6, 2, 2]
1      -1 1     18560 models.common.Conv [32, 64, 3, 2]
2      -1 1     18816 models.common.C3 [64, 64, 1]
3      -1 1     73984 models.common.Conv [64, 128, 3, 2]
4      -1 1    115712 models.common.C3 [128, 128, 2]
5      -1 1    295424 models.common.Conv [128, 256, 3, 2]
6      -1 1    625152 models.common.C3 [256, 256, 3]
7      -1 1   1180672 models.common.Conv [256, 512, 3, 2]
8      -1 1   1182720 models.common.C3 [512, 512, 1]
9      -1 1    656896 models.common.SPPF [512, 512, 5]
10     -1 1    131584 models.common.Conv [512, 256, 1, 1]
11     -1 1         0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
12    [-1, 6] 1         0 models.common.Concat [1]
13     -1 1   361984 models.common.C3 [512, 256, 1, False]
```


Fase de Producción - Iterarion 6



• Conversión modelos TFLITE

```
saved_model_dir = '/content/yolov5/runs/train/yolov5Class3/weights/best_saved_model'

converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16]

tflite_fp16_model = converter.convert()

# Guardar el modelo
with open('/content/best-fp16.tflite', 'wb') as f:
    f.write(tflite_fp16_model)

print(" Modelo float16 exportado como best-fp16.tflite")
```

WARNING:absl:Importing a function (__inference_pruned_11912) with ops with unsaved cu
WARNING:absl:Importing a function (__inference_pruned_11912) with ops with unsaved cu
Modelo float16 exportado como best-fp16.tflite

```
!python detect.py --weights /content/best-fp16.tflite --source /content/IMG_20250304
```

[Mostrar el resultado oculto](#)

```
from PIL import Image
display(Image.open('runs/detect/resultados_fp16/IMG_20250304_193900_box4.jpg'))
```

cantidad	0.88	0.74	3	precios	0.84
cantidad	0.87	0.81	36	precios	0.82
cantidad	0.82	0.72	15	precios	0.77

```
[ ] from PIL import Image
display(Image.open('runs/detect/resultados_fp16/IMG_20250302_125713_box4.jpg'))
```



cantidad	0.86	0.84	14	precios	0.87
cantidad	0.86	0.76	42	precios	0.83
cantidad	0.83	0.30	14	precios	0.81
cantidad	0.87	0.80	42	precios	0.83

```
import tensorflow as tf
interpreter = tf.lite.Interpreter(model_path="/content/best-fp16-v4-class3.tflite")
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
```

```
print("INPUT DETAILS:")
for d in input_details:
    print(d)
```

```
print("\nOUTPUT DETAILS:")
for d in output_details:
    print(d)
```



```
INPUT DETAILS:
{'name': 'serving_default_x:0', 'index': 0, 'shape': array([ 1, 640, 640,  3], dtype=ir

OUTPUT DETAILS:
{'name': 'PartitionedCall:0', 'index': 530, 'shape': array([ 1, 25200,  9], dtype=j
```

Fase de Producción - Iteración 6



• Conversión modelo a TorchScript

```
[ ] model = CRNN(len(vocab))
    model.load_state_dict(torch.load('best_model.pth', map_location='cpu'))
    model.eval()

    example_input = torch.randn(1, 1, 64, 384)
    traced_script_module = torch.jit.trace(model, example_input)
    traced_script_module.save("crnn_model.pt")
```

```
with open('/content/datasetCRNN-Final/vocab.txt', 'r', encoding='utf-8') as f:
    vocab = [line.strip() for line in f]
    char_to_idx = {c: i for i, c in enumerate(vocab)}
    idx_to_char = {i: c for i, c in enumerate(vocab)}
    blank_idx = char_to_idx.get('<blank>', len(vocab) - 1)

#Modelo convertido para cel
model = torch.jit.load("crnn_model.pt", map_location="cpu")
model.eval()
```

```
RecursiveScriptModule(
  original_name=CRNN
  (cnn): RecursiveScriptModule(
    original_name=Sequential
    (0): RecursiveScriptModule(original_name=Conv2d)
    (1): RecursiveScriptModule(original_name=ReLU)
    (2): RecursiveScriptModule(original_name=MaxPool2d)
```

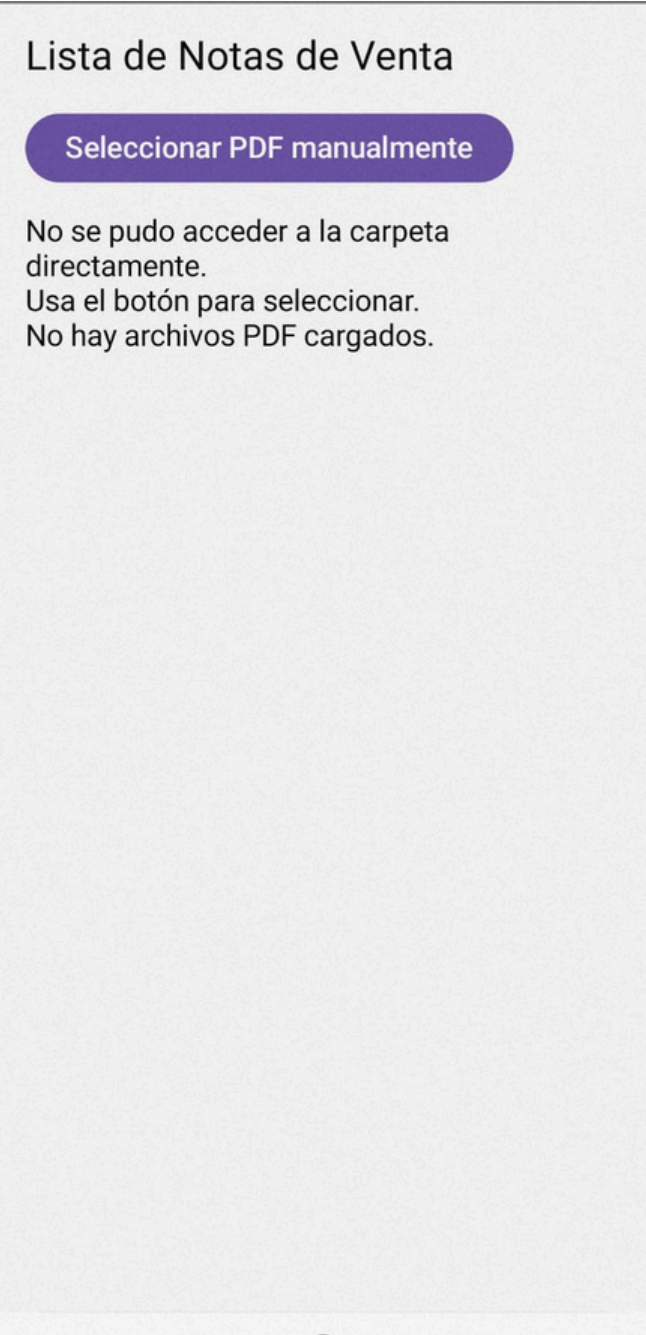
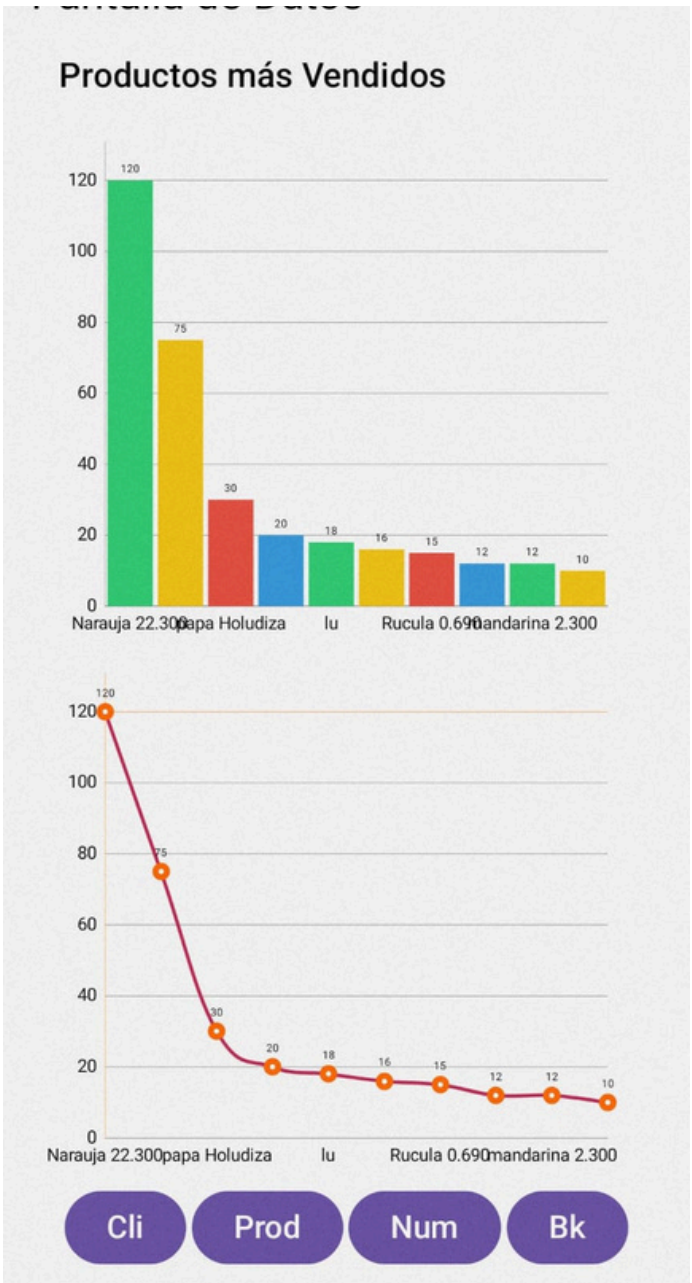
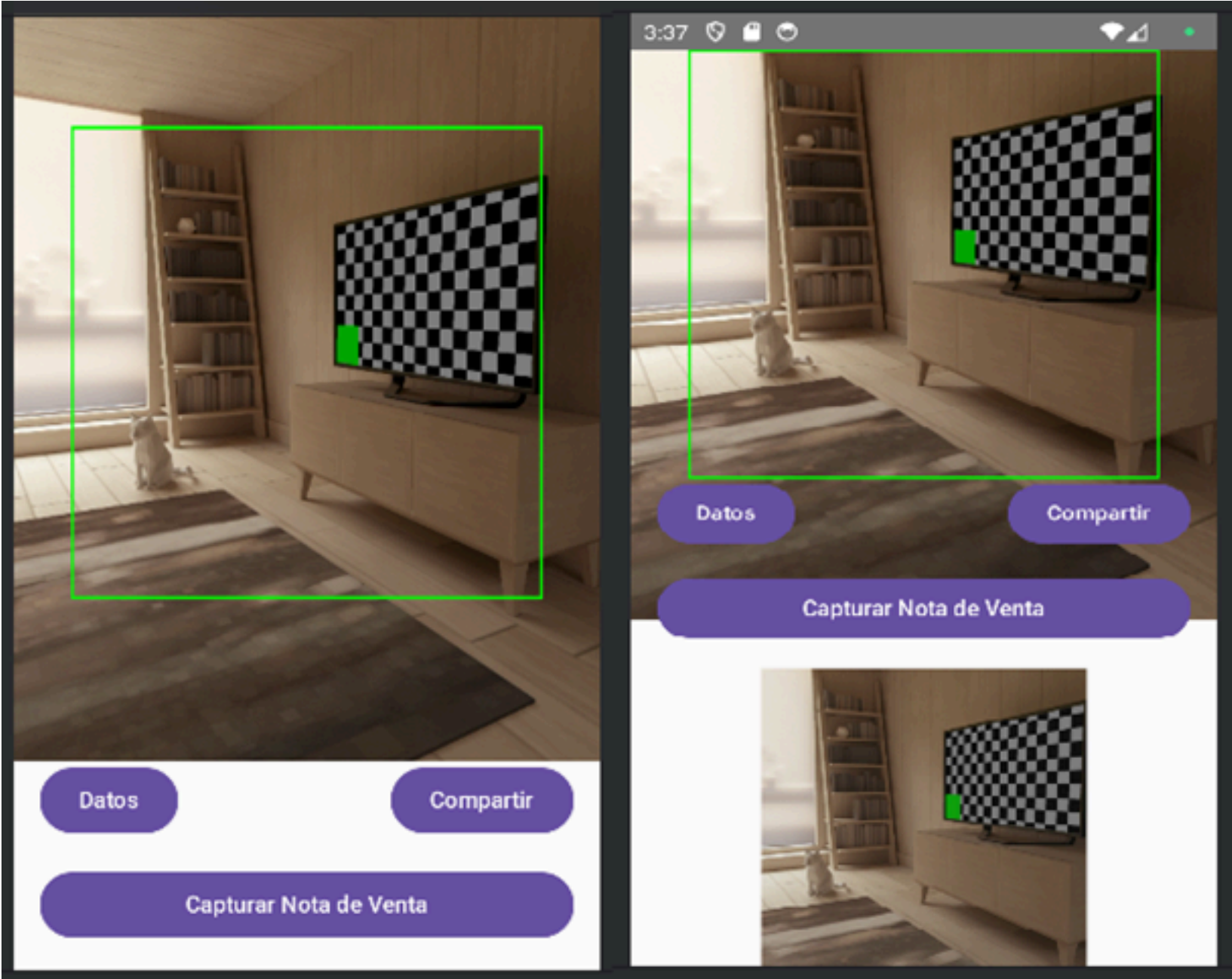
```
test_img_path = '/content/IMG_20250302_125220_box4_63.jpg'
img_tensor = preprocess_image(test_img_path)
with torch.no_grad():
    output = model(img_tensor)
    log_probs = torch.nn.functional.log_softmax(output, dim=2)
    prediction = ctc_greedy_decoder(log_probs)
    print("Predicción:", prediction)
import matplotlib.pyplot as plt
img = cv2.imread(test_img_path, cv2.IMREAD_GRAYSCALE)
plt.figure(figsize=(10, 4))
plt.imshow(img, cmap='gray')
plt.title("Imagen de prueba")
plt.axis('off')
plt.show()
```

Predicción: 36



Fase de Producción - Iterarion 7

- ✓ • Diseño Fronted De La Aplicación.



Fase de Producción - Iteración 7

✓ Implementación Funcional e Integración de Componentes.

```
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = "main") {
        composable("main") { MainScreen(navController = navController) }
        composable("datos") { DatosScreen() }
        composable("compartir") { CompartirScreen() }
    }
}

fun MainScreen(viewModel: MainViewModel = viewModel(), navController: NavController) {
    ) {
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
        Button(onClick = { navController.navigate("datos") }) {
            Text("Datos")
        }
        Button(onClick = { navController.navigate("compartir") }) {
            Text("Compartir")
        }
    }
    //CameraXView(viewModel = viewModel, modifier = Modifier.weight(1f))

    Button(
        onClick = {
            val imageCapture = viewModel.imageCapture ?: return@Button
            val executor = ContextCompat.getMainExecutor(context)
```

```
private val dbHelper = DatabaseHelper(context)
fun insertNota(
    idVenta: String,
    date: String,
    cliente: String,
    productDetails: List<ProductDetail>
): Boolean {
    val db = dbHelper.writableDatabase
    db.beginTransaction()
    try {
        // Generar clave única
        val (increment, letter) = getNextIncrementAndLetter(idVenta)
        val fullSerial = "$idVenta-$increment$letter"

        // Insertar nota de venta
        val noteValues = ContentValues().apply {
            put(DatabaseContract.SaleNoteEntry.COLUMN_FULL_SERIAL, fullSerial)
            put(DatabaseContract.SaleNoteEntry.COLUMN_BASE_SERIAL, idVenta)
            put(DatabaseContract.SaleNoteEntry.COLUMN_INCREMENT, increment)
            put(DatabaseContract.SaleNoteEntry.COLUMN_LETTER, letter)
            put(DatabaseContract.SaleNoteEntry.COLUMN_DATE, date)
            put(DatabaseContract.SaleNoteEntry.COLUMN_REFERRED_TO, cliente)
        }
        val noteResult = db.insert(
            DatabaseContract.SaleNoteEntry.TABLE_NAME,
            null,
            noteValues
        )
        if (noteResult == -1L) return false
        for ((index, detail) in productDetails.withIndex()) {
            val rowNum = index + 1
            val productId = getOrCreateProductId(db, detail.description)
            val detailValues = ContentValues().apply {
                put(DatabaseContract.ProductDetailEntry.COLUMN_SALE_NOTE_SERIAL, fullSerial)
                put(DatabaseContract.ProductDetailEntry.COLUMN_ROW_NUMBER, rowNum)
                put(DatabaseContract.ProductDetailEntry.COLUMN_QUANTITY, detail.quantity)
                put(DatabaseContract.ProductDetailEntry.COLUMN_ID_PRODUCT, productId)
                put(DatabaseContract.ProductDetailEntry.COLUMN_UNIT_PRICE, detail.unitPrice)
                put(DatabaseContract.ProductDetailEntry.COLUMN_TOTAL, detail.total)
            }
            db.insert(
                DatabaseContract.ProductDetailEntry.TABLE_NAME,
                null,
                detailValues
            )
        }
    } catch (e: Exception) {
        db.setTransactionSuccessful()
    }
    db.endTransaction()
    return true
}
```

Fase de Producción - Iteración 7

Implementación Funcional e Integración de Componentes.

```
private const val INPUT_SIZE = 640
private const val NUM_CLASSES = 4 // Tus 4 clases
private const val CONFIDENCE_THRESHOLD = 0.5f // Subir de 0.5 a 0.7
private const val IOU_THRESHOLD = 0.6f // Reducir solapamientos

fun detectarObjetos(bitmap: Bitmap, context: Context): List<Detection> {
    val model = loadModel(context, "best-fp16-v4-class3.tflite")
    val interpreter = Interpreter(model)
    val inputBuffer = preprocess(bitmap)
    val outputBuffer = TensorBuffer.createFixedSize(intArrayOf(1, 25200, NUM_CLASSES + 5))
    interpreter.run(inputBuffer, outputBuffer.buffer)
    val detections = postprocess(outputBuffer.floatArray, bitmap)
    detections.forEach {
        println(" ${it.className} (${("%.1f".format(it.confidence * 100))}% ) " +
            "en [${it.bbox[0].toInt()}, ${it.bbox[1].toInt()}, ${it.bbox[2].toInt()}]")
    }
    interpreter.close()
    return detections
}
```

```
fun loadModel(context: Context): Module {
    return Module.load(assetFilePath(context, "crnn_model-copy.pt"))
}

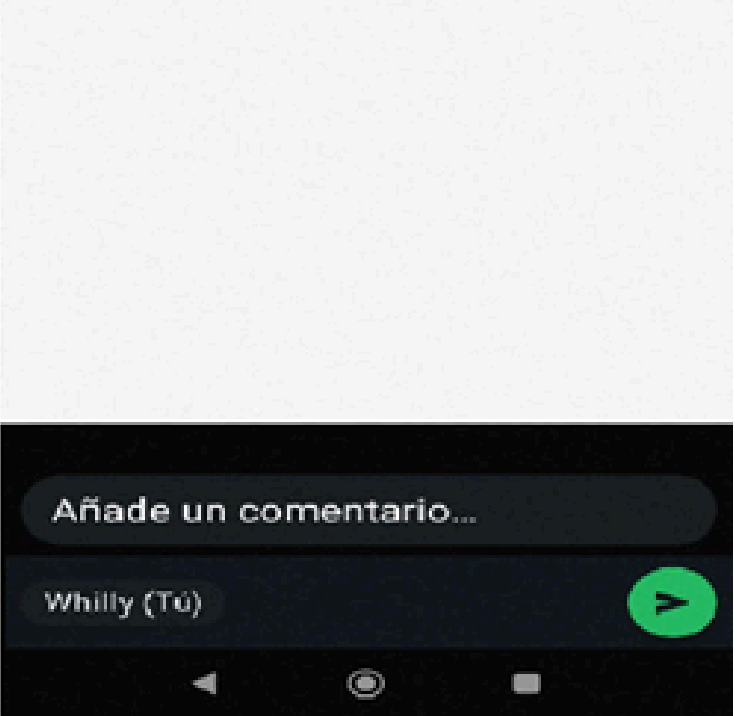
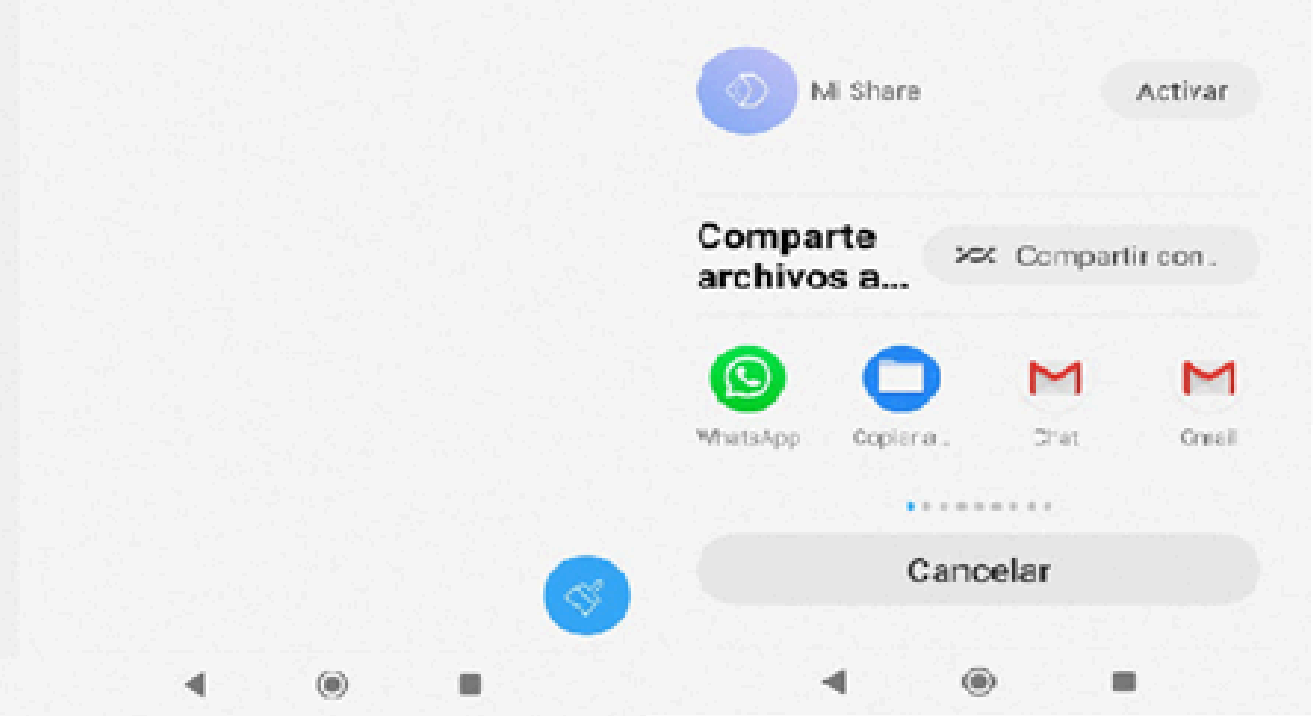
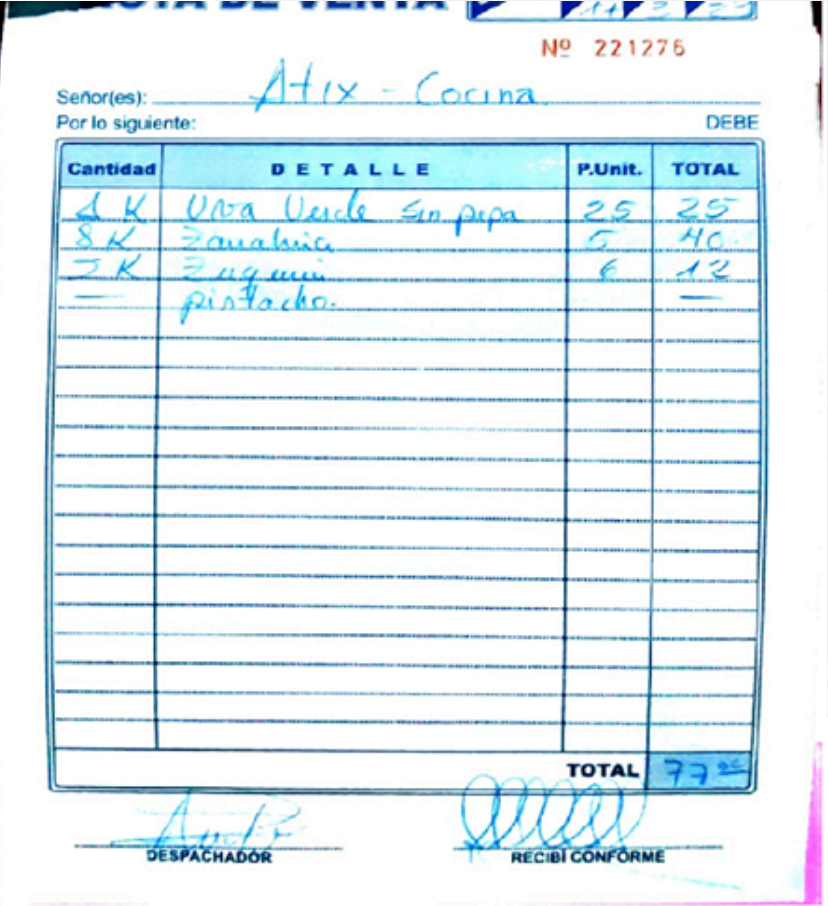
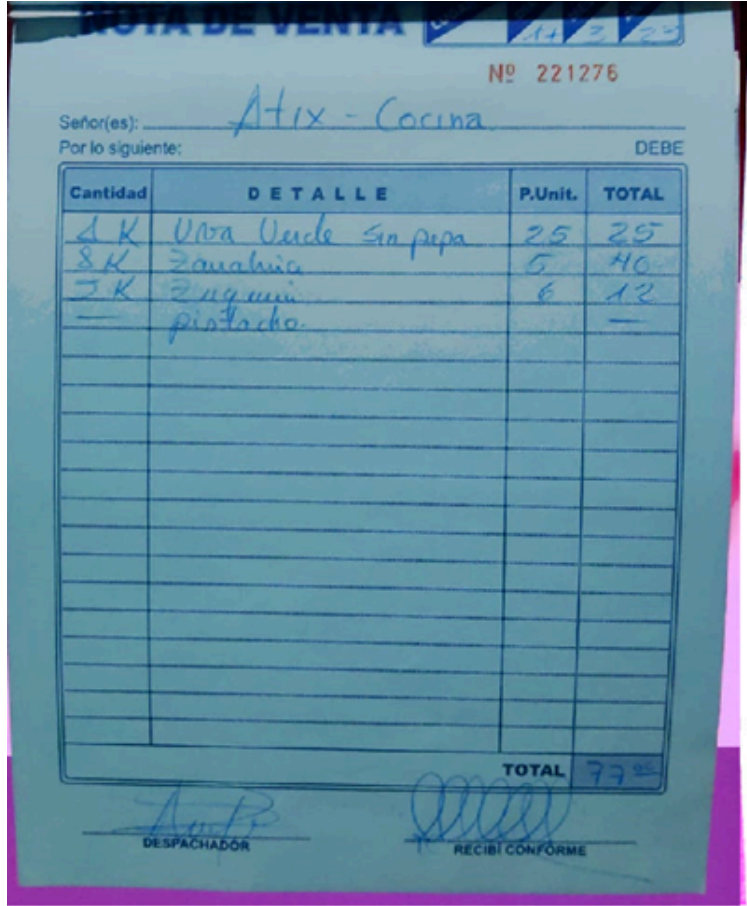
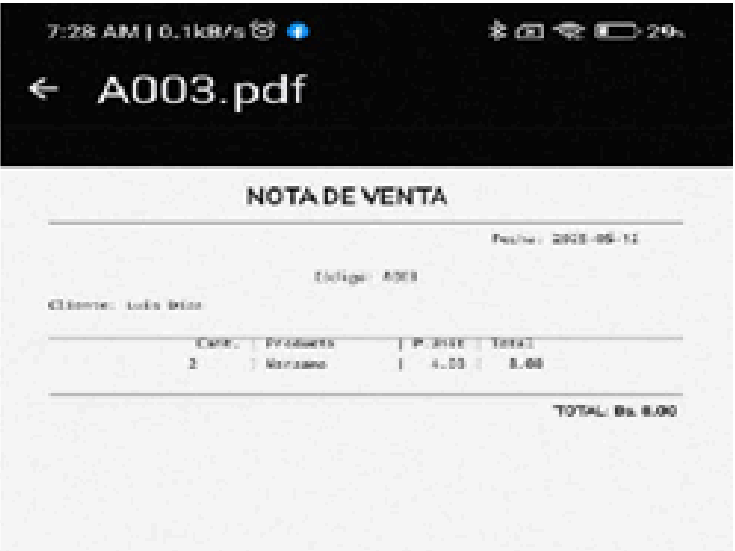
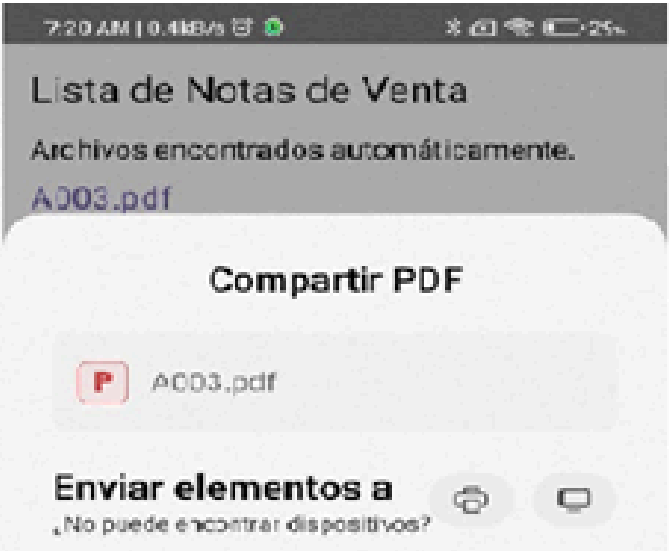
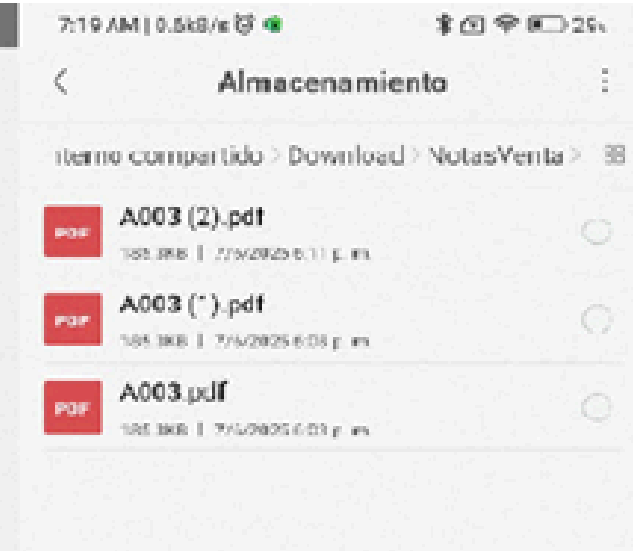
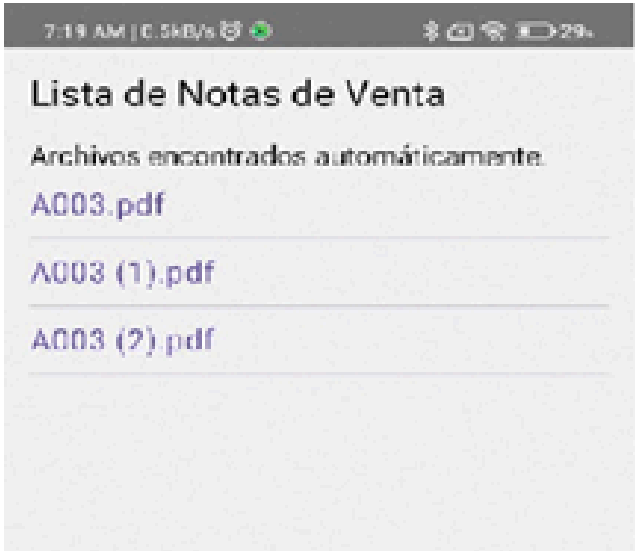
fun predict(context: Context, bitmap: Bitmap, model: Module, vocab: List<String>): String {
    return try {
        val inputTensor = preprocessImageGrayscale(context, bitmap)
        val output = model.forward(IValue.from(inputTensor)).toTensor()
        val outputData = output.dataAsFloatArray
        val shape = output.shape()
        Log.d("Predict", "Output shape: ${shape.joinToString(", ")}")
        Log.d("Predict", "Output data length: ${outputData.size}")

        val I = shape[0].toInt()
        val vocabSize = shape[2].toInt()

        if (!vocab.contains("<blank>")) {
            Log.e("Predict", "Vocab no contiene token <blank>")
            return "Error: vocab no contiene <blank>"
        }

        val predictions = mutableListOf<Int>()
        var lastChar = -1
        for (t in 0 until I) {
            var maxIdx = 0
            var maxProb = Float.NEGATIVE_INFINITY
            for (v in 0 until vocabSize) {
                val idx = t * vocabSize + v
                if (outputData[idx] > maxProb) {
                    maxProb = outputData[idx]
                    maxIdx = v
                }
            }
            predictions.add(maxIdx)
        }
        if (maxIdx != lastChar && maxIdx != vocab.indexOf("<blank>")) {
            lastChar = maxIdx
        }
    } catch (e: Exception) {
        Log.e("Predict", "Error: ${e.message}")
        return "Error: ${e.message}"
    }
}
```


Fase de Estabilizacion





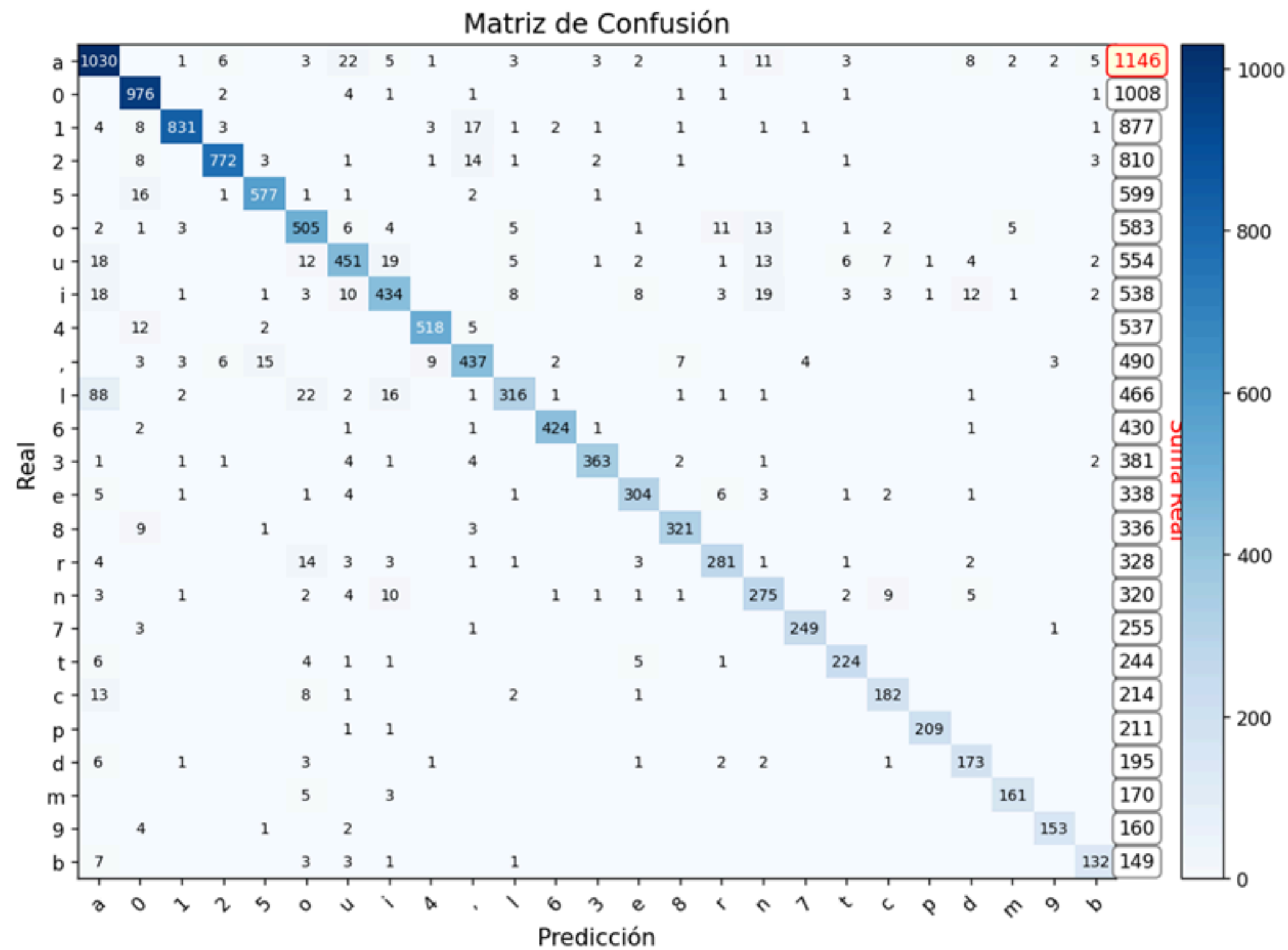
EVALUACION Y RESULTADOS

CAPITULO IV



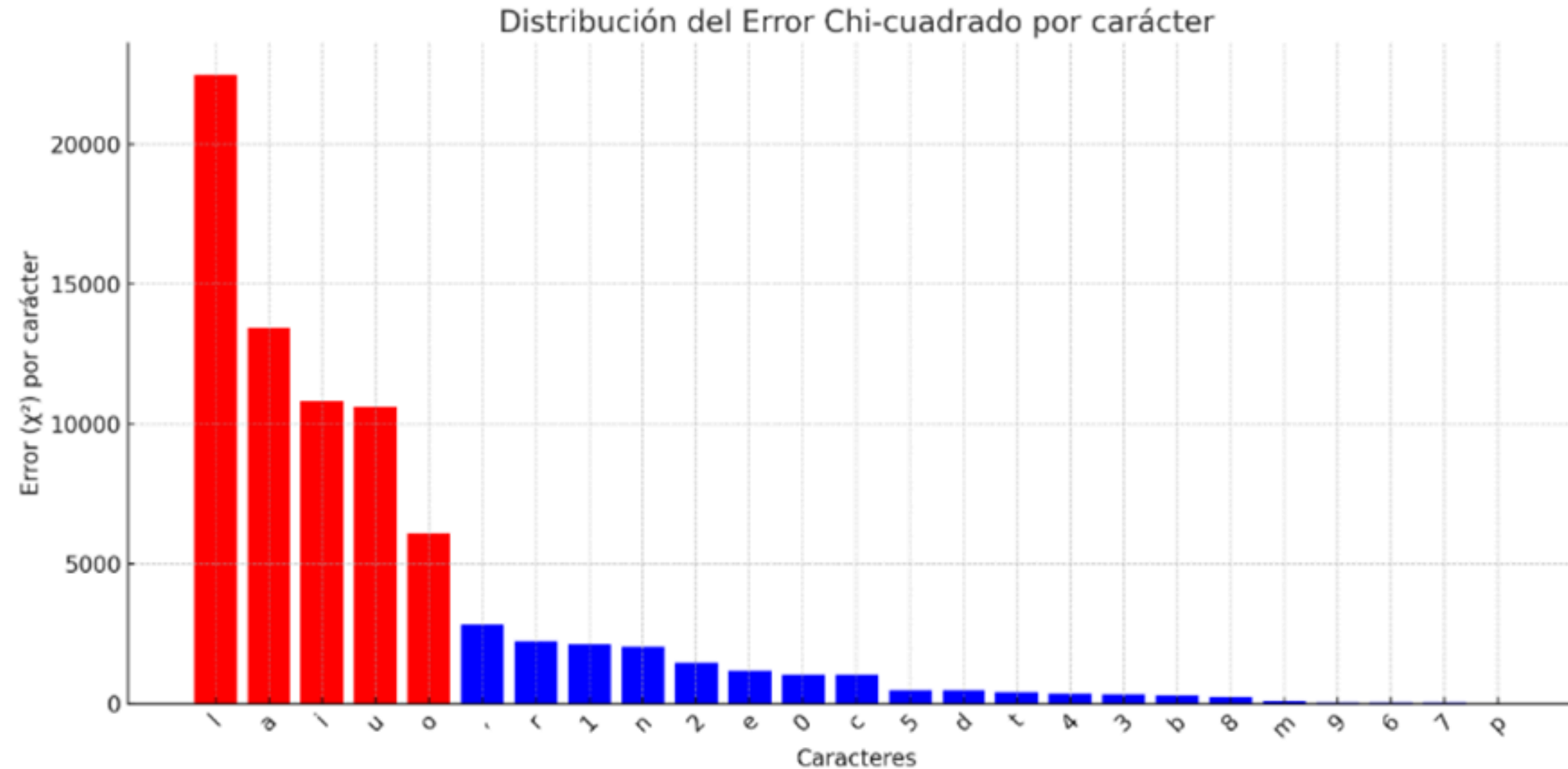
Matriz de confusion

- H_0 : La aplicación móvil basada en Deep Learning y HTR no logra una precisión superior al 70% en la digitalización de notas de venta manuscritas para pequeños comerciantes.
- H_1 : La aplicación móvil basada en Deep Learning y HTR logra una precisión superior al 70% en la digitalización de notas de venta manuscritas para pequeños comerciantes.



Chi-Cuadrado

- ✓ • Cálculo del Chi-Cuadrado de Bondad de Ajuste
- χ^2 valor calculado=185,645477 > χ^2 valor critico 75,6893



Cálculo del Coeficiente Kappa de Cohen



$$OA = \frac{11519}{12729} = 0,904941472 = P_o$$

$$P_e = \sum_{i=1}^k \left(\frac{\text{total real } i}{N} * \frac{\text{total predicho } i}{N} \right) = 0,343307911$$

$$\mathcal{K} = \frac{P_o - P_e}{1 - P_e} = \frac{0,904941472 - 0,343307911}{1 - 0,343307911} = 0,848693663797$$

Valor de κ	Interpretación
< 0	Peor que el azar
0.00 – 0.20	Concordancia leve
0.21 – 0.40	Concordancia aceptable
0.41 – 0.60	Concordancia moderada
0.61 – 0.80	Concordancia sustancial
0.81 – 1.00	Concordancia casi perfecta



 Modelos HTR no comerciales (académicos o de código abierto)

Sistema / Modelo	Precisión manuscrita (%)	Observaciones
IAM HTR baseline (TensorFlow CRNN)	78–85%	Entrenado con el dataset IAM de escritura inglesa, caligráfica. Rendimiento base para líneas completas.
HTR-United (Transkribus, PyLaia, etc.)	83–87%	Modelos entrenados en manuscritos históricos y modernos, depende mucho del corpus y preprocesamiento.
READ project (EU)	80–88%	Excelente rendimiento con estructuras y guías de línea, usado en archivos históricos.
Kraken (OCR/HTR open source)	76–83%	Código abierto, especializado en textos históricos y escritura libre.

Comparacion



Amazon Textract (manuscrita informal)*	78–83%	Precisión en formularios y notas en inglés, depende del idioma y calidad.
Google Cloud Vision OCR (manuscrita)*	72–80%	Precisión decrece con escritura desordenada o informal.
Microsoft Azure Form Recognizer*	76–82%	Alto rendimiento en documentos estructurados, baja en escritura libre.
Abbyy FlexiCapture (OCR profesional)*	80–85%	Mejores resultados en impresos o formularios estandarizados.

CONCLUSIONES



MUCHAS GRACIAS!

Por su atención.



FUNCIONES DE ACTIVACION

TIPO	FUNCION	FORMULA	USADO EN:	VENTAJAS
NL	ReLU	$f(x) = \max(0, x)$	CRNN, YOLOv5	Simplicidad, entrenamiento rápido
NL	Leaky ReLU	$f(x) = x \text{ si } x > 0, \alpha x \text{ si } x \leq 0 \ (\alpha \approx 0.01)$	YOLOv5	Evita muerte de neuronas, mejor que ReLU a veces
NL	Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	LSTM (CRNN)	Salida centrada en 0, buena para secuencias
NL	Sigmoid	$1 / (1 + e^{-x})$	LSTM (CRNN)	Probabilidad entre 0 y 1 (puertas en LSTM)
NL	Softmax	$f(x_i) = e^{(x_i)} / \sum e^{(x_j)}$ (para todos los j)	CRNN, Tesseract OCR	Convierte salidas en probabilidades para clasificación